Binary Arithmetic Operations: Multiplication and Division

1 Binary Division: Restoring Division Algorithm

Overview

Binary division using the **Restoring Division** algorithm mimics long division. It uses repeated subtraction and conditional restoration to compute quotient and remainder.

Key Concepts

- The dividend is stored in the remainder register initially.
- The divisor is subtracted from the remainder.
- If the result is negative, restore the previous value (i.e., undo the subtraction), and shift quotient bit as `0`.
- If the result is non-negative, keep the result and shift quotient bit as `1`.
- The divisor is fixed, but remainder and quotient are shifted.

How Many Steps?

- The number of steps equals the number of bits in the **quotient register** or **dividend**.
- Each step includes: left-shifting, subtracting divisor, and restoring if needed.
- If using an `n`-bit architecture, `n` iterations are required.

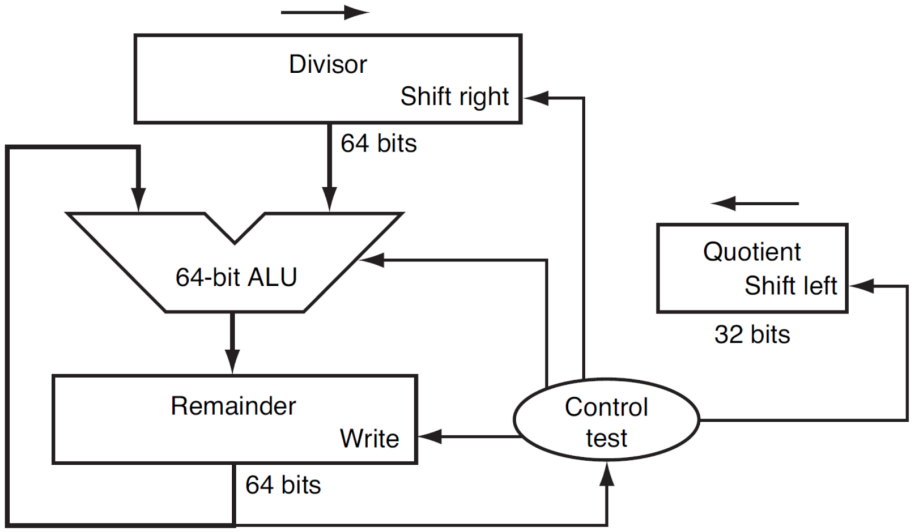
Divisor Alignment and Shifting

- The divisor must be aligned left such that its MSB aligns with the dividend's MSB for initial subtraction.
- For an `n`-bit dividend and `m`-bit divisor, left shift the divisor by $n - m$ bits.
- In restoring division, after each step the divisor remains fixed while the partial remainder is shifted left.

Example: $7 \div 2$

$$\text{Dividend } (7) = 0000\ 0111$$
$$\text{Divisor } (2) = 0010\ 0000 \quad \text{(Left-shifted to match bits)}$$

Division Hardware



Iteration	Quotient	Divisor	Remainder
0 (Init)	0000	0010 0000	0000 0111
1	0000	0001 0000	0000 0111 (Restored)
2	0000	0000 1000	0000 0111 (Restored)
3	0000	0000 0100	0000 0111 (Restored)
4	0001	0000 0010	0000 0011
5	0011	0000 0001	0000 0001

Final Result:

- Quotient = `0011` (3 in decimal)
- Remainder = `0000 0001` (1 in decimal)

$$\boxed{7 \div 2 = 3 \text{ remainder } 1}$$

Another Example: $13 \div 3$

$$\text{Dividend } (13) = 0000\ 1101$$
$$\text{Divisor } (3) = 0011\ 0000$$

(Perform similar steps as above; expect Quotient = 4, Remainder = 1)

2 Binary Multiplication: Shift-and-Add Method

Overview

This method multiplies two binary numbers using repeated addition and bitwise shifting.

Key Concepts

- If the least significant bit (LSB) of the multiplier is `1`, add the multiplicand to the product.
- Shift the multiplicand left (i.e., multiply by 2).
- Shift the multiplier right (i.e., divide by 2).
- Repeat for each bit in the multiplier.

How Many Steps?

- For n-bit multiplier, we perform n iterations.
- Each iteration checks one bit of the multiplier.
- Architecture may optimize using Booth's algorithm or carry-save addition in hardware.

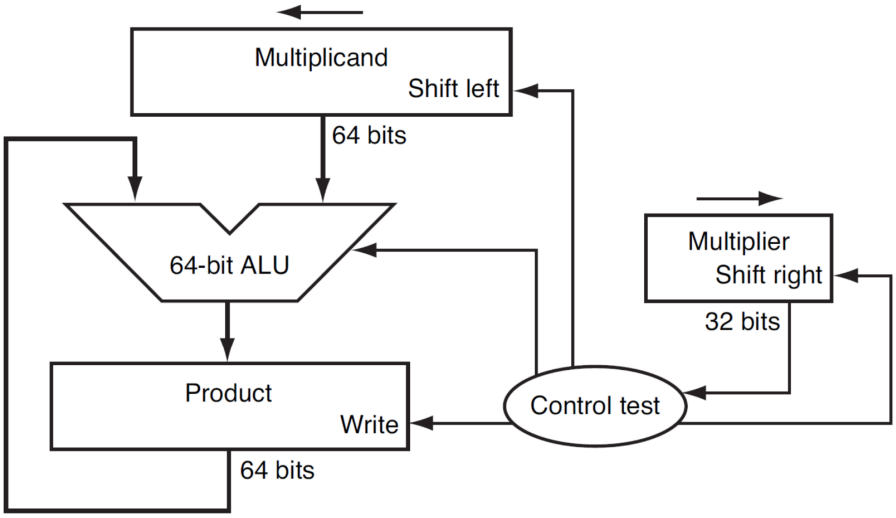
Bit Size Considerations

- For an **n-bit architecture**:
  - The multiplier is typically $\leq n/2$ bits
  - Multiplicand can be $\leq n/2$ bits
  - Product register must be $\leq n$ bits or more to avoid overflow
- Example: In 8-bit architecture:
  - Multiplicand = 4 bits, Multiplier = 4 bits $\rightarrow$ Product = 8 bits
- For 16-bit architecture:
  - Multiplicand = 8 bits, Multiplier = 8 bits $\rightarrow$ Product = 16 bits

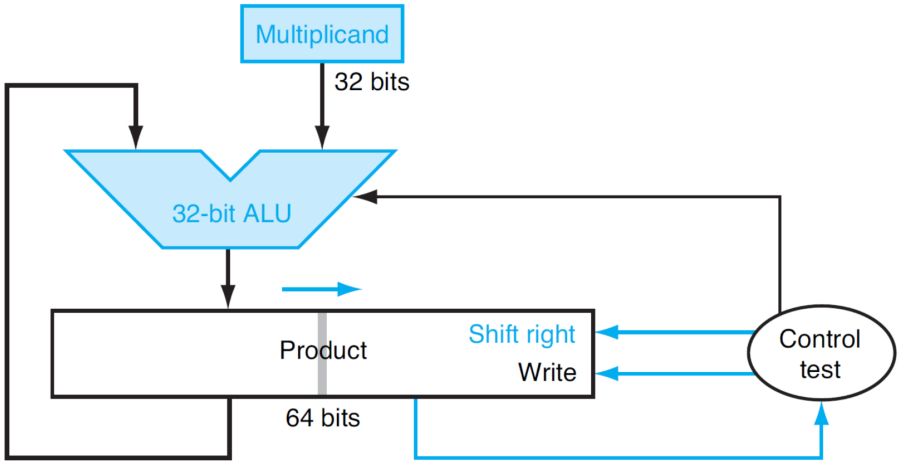
Example: 2×3

$$\begin{aligned} \text{Multiplicand } (2) &= 0000\ 0010 \\ \text{Multiplier } (3) &= 0011 \\ \text{Product (Init)} &= 0000\ 0000 \end{aligned}$$

Sequential Multiplication Hardware



Refined Version of Multiplication Hardware



Iteration	Multiplier	Multiplicand	Product
0	0011	0000 0010	0000 0000
1	0001	0000 0100	0000 0010
2	0000	0000 1000	0000 0110
3	0000	0000 1000	0000 0110
4	0000	0000 1000	0000 0110

$$\boxed{2 \times 3 = 6}$$

Another Example: 5×6

$$\text{Multiplicand } (5) = 0101$$
$$\text{Multiplier } (6) = 0110$$

- Iteration 1: LSB $= 0 \Rightarrow$ No add; shift
- Iteration 2: LSB $= 1 \Rightarrow$ Add multiplicand (5); shift
- Iteration 3: LSB $= 1 \Rightarrow$ Add shifted multiplicand; shift
- Iteration 4: LSB $= 0 \Rightarrow$ No add; final result

$$\boxed{5 \times 6 = 30}$$

3 Hardware Considerations

- Division and multiplication require dedicated circuits like ALUs or separate hardware blocks.
- **Restoring division** is easier to implement but slower.
- Modern CPUs may use **non-restoring** or **SRT** division for performance.
- Multipliers often use **Booth's algorithm** or **Wallace Trees** for speed.
- Bit width (e.g., 8-bit, 32-bit) determines how many shifts and how large intermediate values can be.